

OPERATING SYSTEMS
UE25CS242A

Principles of concurrency
Synchronization Hardware

Dr. Anitha K
Department of Computer Science and Engineering

Acknowledgement, Prof. Pavan A C

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:

1. "Operating System Concepts", Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. "Advanced Programming in the Unix Environment", Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from "Operating Systems, Internals and Design Principles", William Stallings, 9th Edition, Pearson, 2018.

OPERATING SYSTEMS

Principles of concurrency
Synchronization Hardware

Dr. Anitha K
Department of Computer Science and Engineering

Review of Peterson's solution

Code for process i

```
do{
flag[i]=TRUE
turn=j
while(flag[j]&&turn==j);//Do-nop
   critical section
flag[i]=FALSE;
Reminder section
}while(TRUE)
```

Code for process j

```
do{
flag[j]=TRUE
turn=i
while(flag[i]&&turn==i);//Do-nop
   critical section
flag[j]=FALSE;
Reminder section
}while(TRUE)
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Observations

- Atomic Instructions: Software solutions like Peterson's are difficult on modern architectures because CPUs often reorder instructions for performance.

- Solution: HW assisted Synchronization mechanisms

PES UNIVERSITY
CELEBRATING 50 YEARS

Principles of concurrency

- **Principles of Concurrency**
  - Relative speed of execution of processes is not predictable.
  - System interrupts are not predictable
  - Scheduling policies may vary

PES UNIVERSITY
CELEBRATING 50 YEARS

Synchronization Hardware

- Software based solutions are not guaranteed to work on modern computer architectures.
- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking.**
  - Protecting critical regions via locks.
- Synchronization can be done through Lock & Unlock technique.
- Locking part is done in the Entry Section. After locking the process enter critical section.
- The process is moved to the Exit Section after it is done with execution in CS.
- Unlock is done in exit section.
- This process is designed in such a way that all the three conditions of the Critical Sections are satisfied.

Synchronization Hardware



- Uniprocessors – could disable interrupts
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
    - Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
    - **Atomic** = non-interruptible
  - Either test memory word and set value
  - Or swap contents of two memory words

Atomic Operations

- An operation that always runs to completion, or not at all.
- It is **indivisible**: it cannot be stopped in the middle, and state cannot be modified by someone else in the middle.

- Fundamental building block - If no atomic operations, then have no way for threads to work together.

PES UNIVERSITY
CELEBRATING 50 YEARS

Atomic Operations

- On most machines, memory references and assignments (i.e. loads and stores) of words are atomic.
- Consequently – weird examples with partial writes to ints can't happen.
- Many instructions are not atomic.
  - Double-precision floating point store often not atomic
  - VAX and IBM 360 had an instruction to copy a whole array

PES UNIVERSITY
CELEBRATING 50 YEARS

Another Concurrent Program Example



Two threads, A and B, compete with each other

Thread A

```
i = 0;
while (i < 10)
   i = i + 1;
printf("A
wins!");
```

Thread B

```
i = 0;
while (i > -10)
   i = i - 1;
printf("B wins!");
```

- Assume that memory loads and stores are atomic, but incrementing and decrementing are not atomic.

Definitions

- **Synchronization** - Using atomic operations to ensure cooperation between threads.
- **Mutual Exclusion** - Ensuring that only one thread does a particular thing at a time.
- **Critical Section** - Piece of code that only one thread can execute at once. Only one thread at a time will get into this section of code.

PES UNIVERSITY
CELEBRATING 50 YEARS

Locks

- Prevents someone from doing something.
  - **Lock()** before entering critical section and before accessing shared data.
  - **Unlock()** when leaving, after accessing shared data.
  - **Wait** if locked.

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

API for Locks

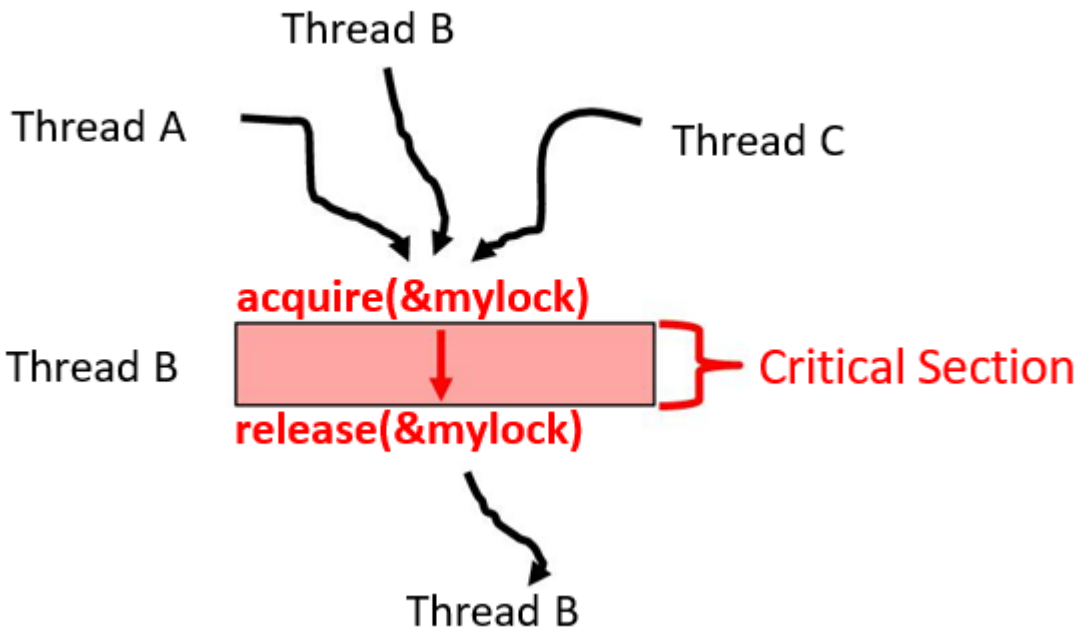


- Locks need to be allocated and initialized:
  - struct Lock mylock or pthread_mutex_t mylock;
  - lock_init(&mylock) or mylock = PTHREAD_MUTEX_INITIALIZER;

- Locks provide two atomic operations:
  - acquire(&mylock) – wait until lock is free; then mark it as busy
  - release(&mylock) – mark lock as free

Note - Should only be called by a process/thread that currently holds the lock

Fix Race Condition problem with Locks



- Threads serialized by lock through critical section.
- Only one thread at a time.

How to Implement Locks?

PES UNIVERSITY
CELEBRATING 50 YEARS

- Atomic instruction sequences
  - These instructions read a value and write a new value atomically
  - Hardware is responsible for implementing this correctly

- on both uniprocessors (not too hard)
- and multiprocessors (requires help from cache coherence protocol)

test_and_set Instruction

PES UNIVERSITY
CELEBRATING 50 YEARS

- Test and Set Lock (TSL) is a synchronization mechanism.
- It uses a test and set instruction to provide the synchronization among the processes executing concurrently.
- It is an instruction that returns the old value of a memory location and sets the memory location value to 1as a single atomic operation.
- If one process is currently executing a test-and-set, no other process is allowed to begin another test-and-set until the first process test-and-set is finished.

test_and_set Instruction



Definition:

```
boolean test_and_set (boolean *target)
  {
      boolean rv = *target;
      *target = TRUE;
      return rv:
  }
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.

Solution using test_and_set()

- Solution:

```
do {
  while (test_and_set(&lock))
    ; /* do nothing */
      /* critical section */
  lock = false;
      /* remainder section */
} while (true);
```

Definition:

```
boolean test_and_set (boolean
*target)
  {
     boolean rv = *target;
     *target = TRUE;
     return rv:
  }
```

compare_and_swap Instruction



Definition:

```
int compare _and_swap(int *value, int expected, int new_value) {
   int temp = *value;

   if (*value == expected)
      *value = new_value;
 return temp;
}
```

```
do{
while(compare_and_swap(&lock,0,1)
!=0);
Critical section
lock=0
Remainder section
}while(true)
```

compare_and_swap Instruction

Definition:

```
int compare _and_swap(int *value, int expected, int new_value) {
  int temp = *value;

  if (*value == expected)
    *value = new_value;
 return temp;
}
```

```
do{
while(compare_and_swap(&lock,0,1)
!=0);
Critical section
lock=0
Remainder section
}while(true)
```

1. Executed atomically
2. Returns the original value of passed parameter “value”
3. Set the variable “value” the value of the passed parameter “new_value” but only if *value == expected. That is, the swap takes place only under this condition.
4. In the x86 and Itanium architectures this is implemented as compare and exchange (CMPXCHG) instruction

Solution using compare_and_swap()

- Shared integer "lock" initialized to 0;
- Solution:

```
do {
      while (compare_and_swap(&lock, 0, 1) != 0)
     ; /* do nothing */
    /* critical section */
 lock = 0;
   /* remainder section */
} while (true);
```

- Mutual exclusion is satisfied
- Do not satisfy bounded waiting requirement

PES UNIVERSITY
CELEBRATING 50 YEARS

Bounded-waiting Mutual Exclusion with test_and_set

This test_and_set algorithm satisfies all the critical section requirements

The common data structures are

boolean waiting[n];

boolean lock;

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;

/* critical section */

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;
    else
        waiting[j] = false;

/* remainder section */

} while (true);
```

PES UNIVERSITY
CELEBRATING 50 YEARS



THANK YOU

Dr. Anitha K
Department of Computer Science and Engineering
anithak@pes.edu